
WHITE PAPER · AI OPERATING GOVERNANCE

Confident and Wrong

Challenge protocols for plausible AI output

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Plausible output needs a challenge protocol before it changes a decision or an action that matters.

The hardest AI error to catch is coherent, specific, and wrong in a way that survives a quick read. Because it matches what the reader expected, it gets the least scrutiny. The remedy is to replace “have a human review it” with a five-part challenge protocol. Name the decision the output feeds. Look for evidence against it. Check the claims it rests on against outside sources. Give review to a named person with the time, expertise, and authority to say no. Learn from each failure.

- 01** Sort output before review by how much its decision matters, so scrutiny goes where depending on the output is riskiest.
- 02** Check the claims a decision rests on against sources outside the answer. Asking the model whether it is sure just consults the same witness again.
- 03** Track how often independent checks change a claim: that rate is the workflow’s real error rate, and it is almost always higher than anyone expected.

The error that survives a quick read

Fluent AI output can enter work that matters carrying more confidence than evidence. A challenge protocol replaces vague advice to verify with five steps. Name the decision at stake. Look for evidence against the answer. Check its claims outside it. Record who reviewed it. Learn from what the review caught.

The generated analysis that came closest to reaching one of my steering decks looked excellent. Clean structure. Plausible figures. A benchmarking comparison that flattered our position, and a citation to an industry report I recognized. One number felt slightly convenient, so I opened the report. The report existed. The number did not. The model had blended two adjacent statistics into one that said what the narrative needed. Nothing in the document gave that away. The only signal was that it agreed with us a little too well, and that week I had the time to be suspicious.

That is the governance problem in miniature. The hardest AI error to catch is not nonsense; nonsense filters itself. It is the answer that is coherent, specific, and

shaped to what the reader expected — and wrong in a way that survives a quick read. Plausibility makes trusting it easy. Agreement makes it easier still. An answer that confirms what someone already thinks gets less scrutiny than one that argues back, so the checking effort lands exactly where it is least needed.

This is the documented failure mode of the technology, not an edge case. A survey of natural-language generation research finds systems that are fluent and coherent yet prone to hallucinate unfaithful or nonsensical content.[3] Where researchers have measured that rate in a domain with real consequences, it runs high. Asked specific, verifiable questions about random federal court cases, leading models hallucinated on a majority of them — 58 to 88 percent depending on the model. They also often failed to correct a user’s mistaken premise, or to recognize when they were hallucinating.[6]

“Have a human review it” is the stock answer, and as stated it is a hope with a name attached. Review that works has structure. It knows what decision the output feeds. It hunts for what would disprove the answer instead of polishing it. It follows claims back to sources. It is done by someone with the time and the standing to say no. It also leaves a trace the organization can learn from. None of that runs on care alone. Users are not credulous and models are not malicious, but at volume the least scrutiny still lands on the most agreeable answers. That has to be met with a control.

This paper turns that structure into a five-part challenge protocol. It is written for AI program owners, risk leaders, workflow owners, and executives who depend on generated analysis. OpenAI’s public account of a sycophancy incident and Microsoft’s lessons from red-teaming a hundred generative products both show the behavior failing in use, not just in theory.[1][2] The protocol itself is what my own reviews hardened into after enough convenient numbers.

Why ordinary review misses this error

Reviewers are trained by human failure, and generated output fails differently. When a colleague’s analysis is weak, the weakness usually shows in how it reads: gaps in the logic, hedged language, formatting that betrays hurry. Fluent generation reverses every one of those signals. The prose is confident, the structure is textbook, the hedges are gone — and none of that tracks with whether the answer is right. A reviewer spends a career learning that polish predicts care. Here that lesson misleads.

Human-factors researchers have a name for the pull this creates: automation bias, the tendency to over-trust an automated aid and under-check its output.[4] In an experiment with 257 medical students, AI explanations lowered diagnostic accuracy

when the AI was wrong, and the students kept following the erroneous recommendations.[5]

The error compounds as people work with it. Someone gets a nearly-right answer and refines it: asks for elaboration, a stronger version, one more polish. Each round makes the document more persuasive without touching the premise underneath. By the third pass, the original unsupported claim is holding up paragraphs built on top of it, and the person who knows the document best is the least able to see the problem. They built their understanding on the same premise the document did.

Volume finishes the job. Generating text is cheap, so output multiplies. Review capacity is human, so it does not. Unless something concentrates scrutiny where the stakes are, the ratio of checking to producing slips quarter by quarter, and nobody notices until an error does real damage. Organizations then typically mandate review of everything. That collapses under its own weight within months, and the organization is back to hope.

The five-part challenge protocol

The protocol is five questions, asked in order, sized to what is at stake. It is deliberately not a checklist for every generated sentence; low-stakes help should stay fast. It is the standard path for output that could change a decision or an action that matters. Each of the five parts exists because a specific, observed failure defeats review without it.

EXHIBIT 1

Five challenges stand between fluent output and a decision

- 1 **Name the decision it could change**
How hard you look should follow what the output may change, not how polished it looks.
- 2 **Try to break the answer**
Test the favored conclusion instead of asking the model to make it sound stronger.
- 3 **Check the claims outside the answer**
Follow the important claims to sources outside the generated answer.
- 4 **Name a reviewer, and a route past them**
Review works only when the reviewer has time, expertise, authority, and somewhere to send a doubt.
- 5 **Learn from the errors you catch**
A rejected answer is evidence about prompts, data, interfaces, review load, and permitted use.

Name the decision it could change

Start by naming what the output might change: which decision, affecting whom, how reversible, at what cost if it is wrong. Until someone states that, the wrong things set how hard people look. Document length. Deadline pressure. How polished the text is. None of those is the stake.

This prevents a specific failure: reviewing a document without knowing it feeds something that matters. A reviewer copy-edits a generated summary as prose. Three weeks later its headline figure anchors an investment recommendation. Nobody decided that figure deserved investment-grade checking, because at review time nobody knew where the summary was headed, and nobody wrote it down. The stakes were set by drift.

The practice is a short use declaration above the output, written before review starts: which decision it affects, who is affected, how reversible it is, what an error costs, and how the output will be used — brainstorming, drafting, a recommendation, or the basis for an action. It takes about a minute. It sorts the output into the right review tier and leaves the record that makes the rest of the protocol checkable later. Low-stakes uses stop here, by design: the first decision the protocol makes is whether the protocol applies.

A workable default has three tiers. Tier one is ideas and internal drafting: no protocol, full speed. Tier two is content feeding a recommendation: look for evidence against it, and spot-check the claims it rests on. Tier three is content that will justify an action affecting money, customers, staff, or the public record: the full protocol, a named reviewer, a verification log. The exact tiers can be adjusted. What matters is that the sorting happens before review, on the stakes someone declared, rather than after an incident, on regret.

Try to break the answer

The instinct with a promising answer is to make it stronger: ask for more support, more polish, more confidence. The second move runs the other way. Try to break it. What observation would make this conclusion false? What other explanation fits the same facts? Who would dispute it, and what is their best argument? Where does the answer hold only inside one framing?

This prevents the echo loop, and sycophancy makes that loop mechanical. OpenAI rolled back a GPT-4o update it described as “overly flattering or agreeable”.^[1] A model that leans that way amplifies whatever framing it is given, so someone who prompts from a preferred conclusion gets better-dressed versions of it. Each refinement feels like progress. The premise is never touched. Teams have gone a dozen rounds deep and ended with what looks like a developed analysis and is the same untested guess, twelve drafts more confident.

Breaking an answer is cheap, and it often uses the same tool. Ask the model for the strongest case against its own answer. Ask what circumstances would make it fail. Ask what data would tell it apart from the next-best explanation. Then check whether any of that was already on hand and simply unexamined. A conclusion earns confidence by surviving a serious attempt to break it. Polishing proves nothing, and the reviewer should be able to say which of the two has happened.

A generated competitive analysis concludes that a rival’s pricing change signals retreat from the mid-market. Three questions, ten minutes. What would retreat predict that we could see — hiring freezes, partners leaving, features dropped — and do we see it? What else explains the pricing change: a bundling strategy, a fiscal-year discount cycle? Who on our own sales team has evidence against the retreat reading? In the case I am describing, the third question ended the thesis. Two account executives had lost mid-market deals to that rival the same month. The analysis had been one prompt away from a strategy meeting.

Check the claims outside the answer

Verification means following the important claims to ground outside the answer. Open the cited report. Re-run the calculation. Read the policy clause itself. Confirm the quotation. Inside the answer, everything is testimony from the same witness. Checking the answer against itself — asking the model whether it is sure, asking the same session for sources — is corroboration theater.

This prevents my convenient-number story and its whole family. Citations to real sources that say something adjacent. Quotations lightly paraphrased into a new meaning. Calculations right in method and wrong in one input. Policy readings that were true two revisions ago. Each one looks like evidence. Each one survives a reviewer who checks that sources are cited without checking what the sources say.

This stays affordable because the target is narrow. Check the load-bearing claims — the ones the decision actually turns on — not every claim. For each, the record is small: source, author, date, the passage or data that supports the claim, and who checked it. That record is what lets a later reader tell what was verified from what was never examined. It is the difference between a document that carries evidence and one that carries formatting.

Two mechanics raise the yield. First, require generated citations in a form someone can open: a working link or a document reference, not a bare title. A citation nobody can open counts as an unsupported claim. Second, keep the verification log beside the document rather than in a separate application. A check that costs a context switch is a check that stops happening in busy weeks.

Name a reviewer, and a route past them

Review is a control only when a named person takes responsibility for what happens to the output. That person needs relevant expertise, protected time, and somewhere defined to send what exceeds their authority. Missing any of those, review is a signature loop: present, recorded, and useless.

The route upward needs one more property: it has to be safe to use. If sending a contested case up reads as incompetence, reviewers will settle their doubts quietly on their own, and the route becomes decoration. The people who receive those cases set the tone in their first few answers. Thank the reviewer, decide the case, and the route stays alive.

This prevents approval in name only, which arrives through three predictable doors. Volume: the approver clears forty items a day, and clicking through becomes the job.

Wrong expertise: the reviewer can judge the prose but not the claim. No authority: the reviewer has doubts and no standing to stop what happens next, so the doubt dies in a comment. Microsoft's red-teaming work makes the general point — finding failure takes people with the expertise and judgment to look for it [2] — and the same holds for the review a workflow runs every day.

The fixes are ordinary management, applied on purpose. Match reviewer tiers to the tiers the use declaration produced. Cap review volume before quality collapses, and treat that cap as a limit on how much gets generated. Separate duties where the requester's interest conflicts with honest review. Define a named route for contested or novel cases. Record what was decided and why, because "approved, with these caveats" is knowledge the organization needs at the next part of the protocol.

Reviewer fatigue is a design input, not a character flaw. Sustained challenge is tiring, and a reviewer who has approved two hundred items in a row has been trained by experience to approve the two hundred and first. Rotate reviewers. Seed known-bad items to keep detection sharp. Inspection disciplines have used both techniques for decades, and they apply here because this is inspection, whatever the interface looks like.

Learn from the errors you catch

A failure someone catches is operating evidence. The blended statistic, the misread policy clause, the calculation with the stale input — each one tells you where the workflow's defenses are thin, which prompts and data invite trouble, and what a given review load is missing. Kept private, the catch protects one document. Fed back, it improves the control for everyone.

This prevents organizational amnesia. Corrections happen inside individual sessions: someone notices, fixes it, moves on. The same failure then repeats across other people and later months, because nothing connects the catches. The organization pays for the same lesson again and again in near misses, and its stated confidence in the workflow never adjusts to any of them.

The practice is light capture with real routing. Keep a failure log: the category, the workflow, where it was caught, and how much was at stake. Review it on a schedule, looking for repeats. Give someone authority to change something when the same failure keeps showing up — prompts, source data, interface defaults, permitted uses, review tiers, or the test set used to evaluate the system. Near misses count double, because a failure caught by luck is a failure the control missed. The test of whether this part works is specific: repeating failure types should stop repeating.

A starter set of categories makes the log useful right away. Fabricated support: sources or figures that do not exist as cited. Blended fact: real pieces combined into a false composite, like my steering-deck visitor. Stale truth: accurate when the model learned or retrieved it, wrong now. Wrong frame: correct information applied to the wrong jurisdiction, segment, or policy version. Induced agreement: the answer bent toward what the prompt visibly wanted. Five categories cover most operating incidents, and where they cluster tells the program where to spend. Fabrication points at verification discipline, stale truth at refreshing the data, induced agreement at prompt and interface design.

The protocol in real use

The five parts cover for each other, which matters because each one alone is easy to escape. A use declaration with no attempt to break the answer produces well-sorted echo. Breaking the answer without checking it outside breaks framings but not fabrications. Outside checks with no real reviewer produce verified inputs to a decision nobody owns. All four without learning leave the organization exactly as exposed next quarter. The strength is the sequence, and the cost control is the first part: most output stops at the declaration, so scrutiny is spent only where the stakes concentrate.

EXHIBIT 2

What each challenge decides**Name the decision it could change**

Apply the protocol when depending on the output could do real damage; keep low-risk help light.

Try to break the answer

Narrow or withhold the conclusion when it survives only inside one framing.

Check the claims outside the answer

Remove the claim or lean on it less when no outside evidence can be found.

Name a reviewer, and a route past them

Send a high-stakes doubt to a qualified owner instead of counting any human touch as review.

Learn from the errors you catch

Reduce or suspend permitted use when failures outrun the organization's ability to catch and correct them.

Two realities deserve honesty. First, the protocol competes with deadlines, and under pressure the temptation is to skip to the signature. The fix is placement: put the questions in the interface or the work record where people actually depend on the output, not in a policy document nobody opens at 4 p.m. Second, the protocol can be faked — declarations pencil-whipped, break-it prompts run and ignored. The fix is sampling: periodic spot-checks that re-verify a handful of decisions, with visible consequences when the record and the reality diverge. Unaudited, the protocol decays into a form.

The protocol is not the same thing as systematic evaluation, and confusing the two wastes both. Evaluation tests the system, using representative cases, thresholds, and re-tests after a change. It answers whether this workflow should depend on this tool at all. The challenge protocol tests one case: this output, feeding this decision, today. Evaluation results set the protocol's tiers — a workflow with strong evaluation evidence can verify by sampling rather than checking everything — and the protocol's failure log hands evaluation its most valuable cases. Evaluation cannot see the one convenient number, and case-by-case review cannot see quality drifting slowly across the whole.

Where to put it, and who owns it

Installation is deliberately small:

- Put the five questions where people actually depend on the output: the interface or the work record.
- Decide which decisions need outside checks and which can be sampled.
- Set reviewer capacity before raising how much gets generated.
- Review repeated failures monthly, and change prompts, data, permissions, workflow, and guidance together.

What that looks like depends on the tool, and the details decide whether anyone adopts it. In a document workflow, the use declaration is a template header and the verification log is a table at the end. In a ticketing or CRM flow, the declaration is two required fields and the reviewer's decision is a status. In a chat interface, the break-it pass is a pinned follow-up prompt the team actually uses. None of this needs new software on day one; it needs the questions placed where the work already happens.

Start with one workflow where the stakes are real and the volume is manageable — contract analysis, customer-facing recommendations, financial summaries — and run the protocol there until it is boring. The early weeks will turn up an uncomfortable number of catches. That is the baseline being discovered, not a new problem. Then expand one workflow at a time rather than by mandate, letting each area set its own tiers and sampling.

Ownership splits cleanly. Workflow owners hold the use declarations and the reviewer rosters. The AI program team holds the failure log and the failures that cross workflows. The risk team holds the sampling audit. The protocol should not grow a bureaucracy; three roles and a monthly review carry it.

What the record shows when it works

A working protocol leaves numbers:

- Output sorted by declared use, and the share landing in each review tier.
- Load-bearing claims checked outside the answer, and how often checking changed the claim.
- Reviewer volume against the caps, and decisions recorded with reasons.
- Failures logged by type, where they were caught, and what was at stake — near misses included.

- Repeating failure types retired by a change to the control.
- Sampled decisions that survived a re-check.

Resist rolling these into a single compliance percentage. One “protocol adherence” score invites exactly the theater the sampling audit exists to catch, and it hides the interesting signals: which kinds of claims keep failing verification, whose decisions keep surviving re-checks, and which workflows keep turning out to matter more than they declared. These numbers are a diagnostic set, and their disagreements are the findings.

Watch the second measure early. How often an outside check changes a claim is the workflow’s real error rate, and it is almost always higher than anyone expected. Watching that rate fall as prompts, data, and guidance improve is the protocol’s return, made visible.

Three objections: distrust, better models, speed

The protocol does not assume every model answer is false, or that a human is automatically right. It adds deliberate checking steps where the stakes warrant it. The safeguards are outside evidence, qualified judgment, and action you can walk back. General-purpose distrust is not a safeguard.

The second objection is that models are improving, so the protocol will age out. That gets the trend right and the conclusion wrong. Better models shrink the error rate. They do not shrink the damage of the errors that remain, and they make those errors more plausible, not less. A control keyed to what is at stake rather than to how often errors occur is exactly the control that survives better models. The tiers stay. Verification volume falls as evidence accumulates. The failure log tells you which check to relax and when. That is the protocol adjusting to better models, not being retired by them.

The third objection is speed. Yes, the protocol slows the last step of work that matters. That is what it is for. Frictionless speed is the wrong benchmark. The real alternative is the incident — the fabricated figure in the board deck, the misread policy in the customer letter — and the blanket review mandate that follows it and slows everything. Checking sized to the stakes, done at the moments that matter, is the cheap version of trust.

What changes when review becomes a control

Once challenge is a control, review stops being a virtue and becomes a mechanism. Fluency stops working as evidence, because the workflow no longer accepts it as any. The confident and wrong answer will keep arriving; that is the nature of the tools. What changes is what it meets: a reader with a quick eye and a deadline, or a protocol built for exactly this visitor.

Notes and sources

[1] OpenAI, "Sycophancy in GPT-4o: What Happened and What We're Doing About It," April 2025. <https://openai.com/index/sycophancy-in-gpt-4o/>

[2] Blake Bullwinkel et al., "Lessons From Red Teaming 100 Generative AI Products," Microsoft Research, January 2025. <https://www.microsoft.com/en-us/research/publication/lessons-from-red-teaming-100-generative-ai-products/>

[3] Ziwei Ji, Nayeon Lee, Rita Frieske, et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, 2023. <https://arxiv.org/abs/2202.03629>

[4] Raja Parasuraman and Dietrich H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," Human Factors 52, no. 3 (2010): 381–410. <https://depositonce.tu-berlin.de/bitstreams/cafd2873-814b-4c59-bab1-add42e249d2/download>

[5] Manshu Khanna, Ziyi Wang, Lijia Wei, and Lian Xue, "When Medical AI Explanations Help and When They Harm," arXiv:2512.08424, 2025. <https://arxiv.org/abs/2512.08424>

[6] Matthew Dahl, Varun Magesh, Mirac Suzgun, and Daniel E. Ho, "Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models," Journal of Legal Analysis 16(1), 2024, 64–93; open version arXiv:2401.01301. <https://arxiv.org/abs/2401.01301>

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.